CIDADES ESG INTELIGENTES

DOCUMENTAÇÃO TÉCNICA • DEVOPS

Automação de build, testes, containerização e deploy sequencial para staging e produção.

Stack	C# • ASP.NET Core .NET 8 • Entity Framework Core • SQL Server
CI/CD	GitHub Actions • GitHub Container Registry • SSH • Docker Compose
Integrantes	Guilherme Luz Martins - RM 564344 Henrique Miranda Paulino - RM 565689 Nicolas Granatelli Sanches - RM 565755 Henry Gois Ribeiro - RM 564329 Matheus Pedrosa Mendes da Costa - RM 564328
Data	13 de outubro de 2026

Validação local em 13/10/2026: restore e build Release concluídos com 0 erros e 0 avisos; 5 de 5 testes xUnit aprovados. Pipeline GitHub, imagem, deploy e capturas de staging/produção ainda dependem da infraestrutura externa.

1. Projeto e arquitetura

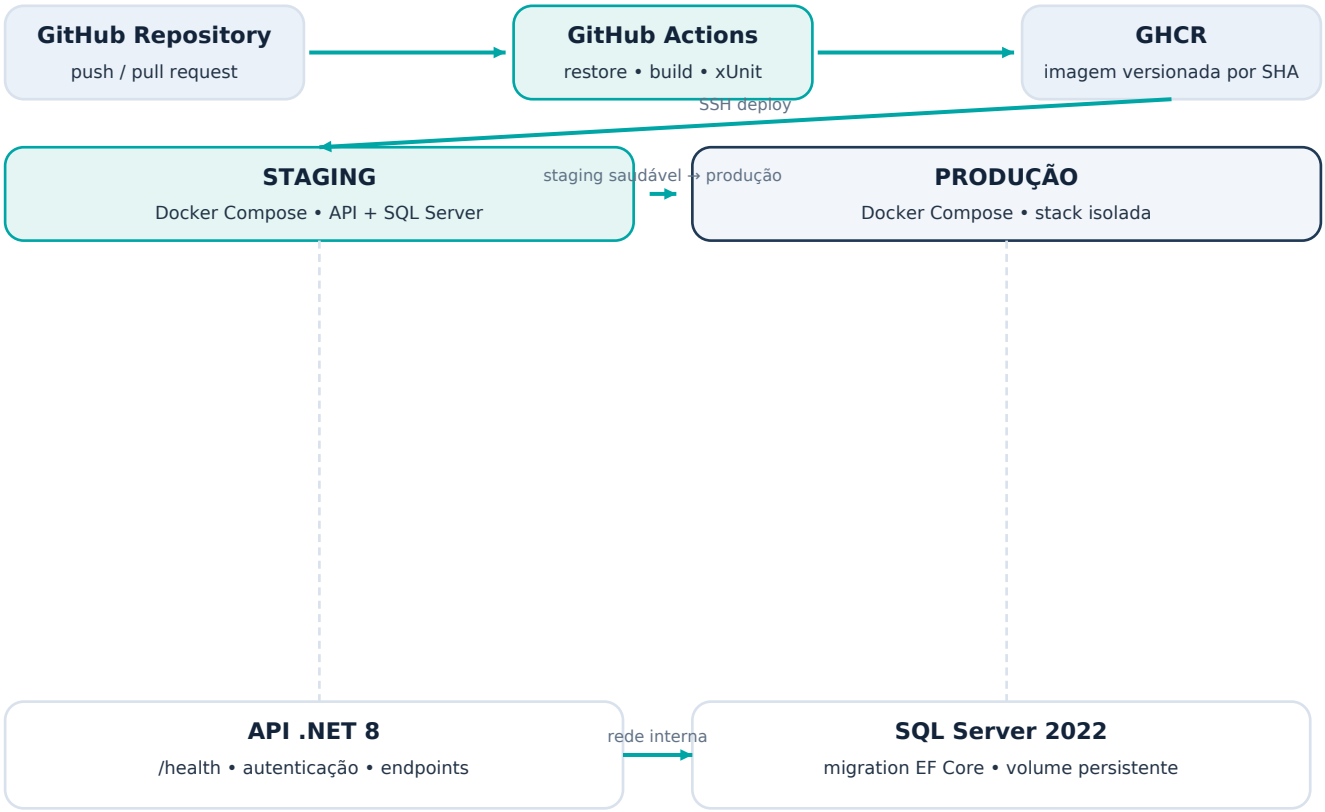
Visão funcional do backend e do fluxo de entrega contínua.

O backend recebido é uma API REST em ASP.NET Core .NET 8. A estrutura existente separa Controllers, Services, Models, ViewModels, persistência via Entity Framework Core e migrations. A solução contém testes xUnit para os endpoints dos controllers.

Funcionalidades existentes

- Consumo energético com operações CRUD e paginação.
- Dashboard com indicadores, rankings de eficiência e dados de consumo.
- Alertas de consumo e automação IoT para dispositivos ociosos.
- Autenticação HMAC Bearer com papéis ADMIN e OPERATOR.

Arquitetura proposta para CI/CD



Cada ambiente Compose tem sua própria instância de API, volume de dados e projeto Compose. A imagem é versionada pelo SHA do commit. O job de produção depende de staging ter terminado com sucesso.

2. Pipeline CI/CD

Arquivo de implementação: `.github/workflows/ci-cd.yml`.

ETAPA	O QUE ACONTECE	GATE / RESULTADO
1. Checkout + SDK	Obtém o código e configura o SDK .NET 8.	Ambiente pronto para build.
2. Restore + build	Restaura pacotes e compila a solução em Release.	Erro interrompe o workflow.
3. Testes xUnit	Executa os testes existentes e publica relatório TRX como artefato.	Testes devem passar para publicar imagem.
4. Build + GHCR	Constrói a imagem com Docker Buildx e publica tags SHA e latest no GHCR.	Somente push para main fora de pull request.
5. Staging	SSH envia Compose e variáveis; servidor faz pull, atualiza containers e espera health checks.	Deploy precisa ficar saudável.
6. Produção	Aplica a mesma imagem exata do commit depois do sucesso de staging.	Bloqueado se staging falhar; pode exigir aprovação.

Configuração externa necessária

- Criar os GitHub Environments `staging` e `production`.
- Definir variables `SSH_HOST`, `SSH_USER`, `SSH_PORT`, `DEPLOY_PATH`, `APP_PORT` em cada ambiente.
- Definir secrets `SSH_PRIVATE_KEY`, `SSH_KNOWN_HOSTS`, `MSSQL_SA_PASSWORD`, `AUTH_SIGNING_KEY`, `GHCR_USERNAME`, `GHCR_READ_TOKEN`.
- Preparar servidores Linux com Docker e Docker Compose v2 e usuário de deploy com permissão de executar Compose.

O workflow não contém senhas ou chaves reais. Sem repositório GitHub, secrets e hosts, os jobs de deploy não podem ser comprovados. A lista completa de configuração está no README.

3. Docker e orquestração

Dockerfile e arquivos Compose incluídos no código-fonte.

Estratégia do Dockerfile

- Build multi-stage: SDK .NET 8 restaura dependências e publica a API em Release.
- Imagem final `aspnet:8.0` reduz o conteúdo de build dentro do runtime.
- Execução com usuário não-root disponibilizado pela imagem .NET e health check HTTP em `/health`.

Serviços do Compose

SERVIÇO	FUNÇÃO	CONFIGURAÇÃO
api	Aplicação ASP.NET Core.	Porta 8080 no container; connection string e chave HMAC por env; aguarda banco saudável.
db	SQL Server 2022 Developer.	Health check `sqlcmd`; volume de dados persistente; rede interna.
esg-network	Rede de serviços.	A API se conecta ao host interno `db:1433`.
sqlserver_data	Persistência de dados.	Não é removido pelo `docker compose down` comum.

Comandos principais

```
# Preparar ambiente (copiar .env.example para .env)
docker compose up --build -d --wait --wait-timeout 180
docker compose ps
curl http://localhost:8080/health
docker compose logs -f api db
docker compose down
```

A opção `Database:ApplyMigrationsOnStartup` habilita migrations automáticas somente quando ativada no ambiente. O código tenta reconectar com número limitado de tentativas antes de falhar a inicialização.

Observação de produção: SQL Server Developer é para desenvolvimento/teste, não para produção comercial. Um serviço real deve usar licença/edição apropriada ou banco gerenciado, backups, TLS via proxy reverso, monitoração e política segura de migrações.

4. Evidências de execução

Substitua cada quadro por uma captura de tela real após executar build, testes e deploy.

Não foram fabricadas evidências: o pipeline ainda precisa ser executado no GitHub e os ambientes dependem de servidores e secrets fornecidos pelo responsável pelo projeto.

01 • PIPELINE BUILD + TESTES

Captura real do GitHub Actions com os jobs verdes.

INSERIR PRINT REAL AQUI

02 • IMAGEM GHCR

Página do pacote com o tag SHA do commit.

INSERIR PRINT REAL AQUI

03 • STAGING

Navegador/Postman/curl mostrando HTTP 200 em /health.

INSERIR PRINT REAL AQUI

04 • PRODUÇÃO

Navegador/Postman/curl mostrando HTTP 200 em /health.

INSERIR PRINT REAL AQUI

Sugestão de nomes dos arquivos: `pipeline-build-test.png`, `ghcr-image.png`, `staging-health.png`, `production-health.png`. Salve-os em `docs/evidencias/`, insira-os no README e atualize este PDF depois da execução.

5. Desafios, soluções e checklist

Resumo da adaptação do backend e do que falta comprovar.

DESAFIO	SOLUÇÃO ADOTADA
SQL Server LocalDB no projeto original	Compose utiliza SQL Server Linux com connection string via variáveis de ambiente; o host do banco dentro da rede é `db`.
API inicia antes do banco	Health check do SQL Server + `depends_on: service_healthy`; migrations são aplicadas com repetição limitada quando habilitadas.
Comprovação de deploy e status	Workflow faz staging antes de produção e aguarda os health checks. É necessário fornecer hosts, keys e secrets e capturar os resultados reais.
Proteção de segredos	Arquivo `.env` ignorado pelo Git; variáveis sensíveis ficam nos GitHub Environments e são enviadas ao servidor com permissões restritas.

Checklist de entrega

ITEM	STATUS ATUAL
Projeto organizado e ZIP gerado	**Preparado**
Dockerfile e Compose	Arquivos incluídos; execução local ainda deve ser validada
Pipeline build/test/deploy	Configurado; aguarda execução no GitHub
PDF técnico	Incluído; capturas reais pendentes
Testes aprovados	5/5 aprovados localmente; pipeline pendente
Deploy staging + produção	Pendente de servidores/secrets e execução
Prints reais anexados	Pendente

Antes da entrega final, informe os integrantes, rode a validação, configure os environments, espere o pipeline concluir, teste os dois endpoints `/health`, insira os prints e gere novamente o ZIP final.